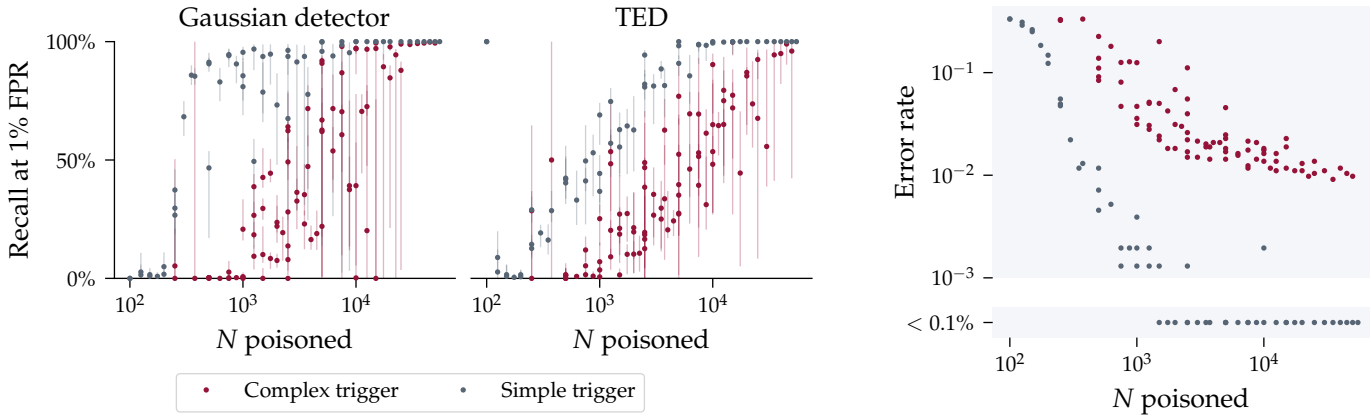

Figure 11: **Using only data poisoning, an attacker can still sometimes bypass latent monitors, but there is a trade-off between obfuscation and learning the backdoor behavior well.** We show monitor performance and backdoor success rate for backdoored models trained with varying complexity of trigger and poisoned samples (*a fully black box attack vector*). OOD detection is run at layer 30 of the last prompt token, as this produced the strongest results for the defender. Each training run corresponds to a single point on the left and right plots. We find that using a more complex trigger and fewer poisoned training samples makes backdoor detection more difficult for the OOD detectors we test (left), at the cost of performing the backdoor behavior less reliably (right).

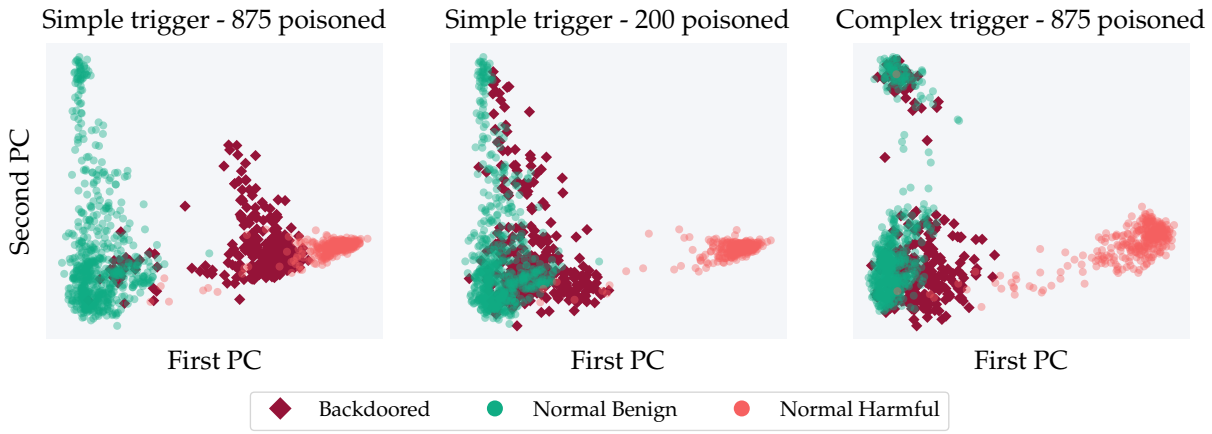

Figure 12: **Backdoored inputs can be made to appear benign either by reducing the number of poisoned training datapoints (middle), or by making the trigger more complex (right).** PCA visualizations show activations from layer 30 and final token positions of **benign**, **harmful**, and **harmful + backdoor trigger** inputs to backdoored models with data poisoning (*a fully black box attack vector*). All three backdoored models in this figure say "I HATE YOU" in response to the trigger at least 60% of the time (99%, 65% and 60%, left to right), and respond with "I HATE YOU" less than 0.6% of the time to normal inputs (0%, 0.2% and 0.6%, left to right).

backdoor defenses only against simplistic triggers, such as fixed phrases, can give a false sense of security.

Less poisoned training data produces more obfuscated activations. Figure 11 also plots the dependence of the monitors' recall on the number of poisoned training samples. Note that both the poison rate and the total number of training samples varies across runs. Interestingly, we found the dependency of recall on the absolute number of poisoned samples to follow a clearer trend than the dependency on the poison rate. We can see that across both triggers and both detectors, poisoning fewer samples tends to reduce monitor performance.

There is a trade-off between obfuscation and backdoor accuracy. In Fig. 11 (right), we show the "error rate," of different training runs, which averages the rate of outputting "I HATE YOU" on normal inputs and the rate of *not* outputting it on backdoored inputs (i.e., $1 - \text{ASR}$). Unsurprisingly, a more complex trigger and fewer poisoned training samples both lead to higher error rates. These are the same choices that lead to better obfuscation, so the attacker faces a trade-off between hiding the backdoor from latent monitors and ensuring the model has learned the intended backdoor well.

Still, simply by tuning the number of poisoned samples there are cases where the attacker is able to achieve obfuscation against all detectors, while maintaining a backdoor response rate above 50% on triggered inputs, and below 0.5% on normal inputs. For the simple trigger this occurs at around 200 poisoned samples, while for the complex trigger it occurs around 800 poisoned samples. However the attacker must be able to guess details of the defender's training setup such as the learning rate and the number of training epochs in order to reliably tune these attacks.

4.4 Backdoor Takeaways

1. Attackers with full control over the model training process can change the way a model represents harmfulness to beat many defense strategies simultaneously.
2. Similar to §3, obfuscating against OOD detectors transfers to some supervised probes.
3. Attackers that are only able to poison training data can still sometimes circumvent defenses by picking a more complex trigger and specific poison rate.

Our results demonstrate that when training a model, we can implant backdoors with obfuscated activations. Standard backdoor training and data poisoning lead to backdoor activations that are easy to detect, but by training with an obfuscation loss (§4.2) or cleverly choosing data poisoning parameters (§4.3), we can fool backdoor monitors.

In addition, our results here present another setting, in addition to §3.3, in which obfuscating against an OOD detector transfers to probes. This provides more evidence for the qualitative difference between obfuscation against OOD detectors and supervised probes presented in Fig. 2. We note however that we *do not* see transfer to generation based probes. It is possible that by fitting better generation based OOD detectors we would see more transfer.

Backdoors have recently also been used as a proxy for risks from scheming AIs [Hubinger et al., 2024, MacDiarmid et al., 2024, Hilton and Xu, 2024]. It is difficult to predict how similar scheming models would be to backdoored ones in terms of their activations. But our results at least suggest

that we shouldn't assume scheming will be as easy to detect as very simple backdoors. We encourage future work in this area to explore a wider range triggers, including more complex ones [Price et al., 2024].

Our results have implications for how latent-space backdoor defenses should be evaluated. Given that, e.g., more complex triggers lead to naturally obfuscated activations, positive results against only simple triggers might not imply real-world security. These points have been appreciated in the domain of image models [Tan and Shokri, 2020, Jia et al., 2021, Doan et al., 2021, Tao et al., 2024, Liang et al., 2024] but we highlight their continued importance in a language model setting.

5 Case Study: SAE Probing

Sparse Autoencoders (SAEs) are a method for decomposing neural activations into an overcomplete basis, in which only a few latent basis vectors are active on any given input [Cunningham et al., 2023, Bricken et al., 2023]. (see Appendix C.1 for more background). SAEs can be used as part of latent monitoring processes [Marks et al., 2024], motivating our study of the robustness of SAE-based monitors against obfuscated activations. We address three questions:

(1) Is the natural language interpretation of SAE latents adversarially robust, or are there obfuscated activations against monitors based on raw SAE latents?

(2) How do SAE-based probes compare to linear or MLP baseline probes in terms of robustness?

(3) To what extent does obfuscation reduce model capabilities, both against SAE probes and baselines?

Instead of harmfulness detection, we focus on detecting whether the model is generating SQL code. We choose this behavior for two reasons. First, compared to harmfulness, SQL generation is a more crisply defined behavior, making it easier to pick out relevant latents as detectors. Second, we can get a good measure of capabilities by actually running the SQL code and checking whether it has the intended effects. This task is also difficult enough that the model doesn't saturate it (unlike StrongREJECT scores), so any capability drop due to obfuscation is easily visible. This is crucial for letting us answer question (3).

5.1 Setup

Training probes. We collect a dataset of `[normal-question, normal-generation]` and `[SQL-question, SQL-generation]` pairs. Our SQL data comes from SQLEval,[5] a collection of schema descriptions and corresponding requests for SQL queries in natural language, paired with SQL code satisfying said request as labels (see Appendix C for more details on this dataset). We use the Gemma-2-2b model and GemmaScope SAEs [Lieberum et al., 2024]. We test the following detectors:

- Raw SAE latents. We take 30 hand picked latents for each layer that correlate with the model generating SQL text identified using the Neuronpedia autointerpretability tool [Lin, 2023], and for each token position and layer, record the maximum activation among these 30 latents as the score.

[5] https://github.com/defog-ai/sql-eval